
Jobs — Market Analysis

Release 1.0

Benoit Chevallier-Mames

Oct 03, 2026

CONTENTS

1	Market Analysis — Self-Hosted Job-Search Aggregators	1
1.1	Executive summary	1
1.2	Methodology	1
1.3	Direct competitors	2
1.4	Commercial job-search CRMs (adjacent category)	3
1.5	Where this repo sits on the market map	4
1.6	Design decisions that are rare in the field	4
1.7	Realistic paths forward	5
1.8	Recommendation	6
1.9	Open questions worth asking the user	6

MARKET ANALYSIS — SELF-HOSTED JOB-SEARCH AGGREGATORS

Author: compiled for Benoit Chevallier-Mames **Date:** October 2026 **Repo under review:** `sandboxed-repos/jobs` (147 companies, Playwright + LLM scoring)

1.1 Executive summary

The self-hosted job-aggregator space is **crowded but fragmented**. There are dozens of open-source scrapers on GitHub, most of them maintained by a single author for their own job hunt. None of them combine all four of the following properties, which is what makes the repo under review distinctive:

Property	Common in competitors?
100+ ATS sources (not just LinkedIn)	Rare
Per-user state (reject/like/apply/K)	Common
LLM-based fit scoring with a profile	Rare
Senior-level focus (SENIORITY_XP, IC-level badges)	Essentially unique

The closest named competitors are **JobSpy** (consumer boards), **ats-scrappers / jobhive** (ATS-native, 50+ platforms), **JobSentinel** (local-first UX, preference-based scoring), and **JobAggregator** (FastAPI dashboard tuned for India + remote AI). Each one dominates a different corner of the design space — none covers the combination this repo targets.

1.2 Methodology

We searched GitHub + the open web (October 2026) for projects tagged `job-scraper`, `job-aggregator`, `job-search`, and `self-hosted`, plus Scrapfly's August 2026 roundup of actively-maintained open-source scrapers. For each candidate we extracted:

- Source coverage (which boards / ATS)
- Deployment model (CLI, FastAPI, Docker, GitHub Pages)
- State tracking (does the user have persistent rejected/liked state?)
- Scoring (preference rules, LLM, nothing)
- Target user (generic job seeker, senior engineer, specific vertical)
- Last commit (as a proxy for whether the project still works)

Projects with no commits in the past 6 months were excluded unless they scored exceptionally high on another axis.

1.3 Direct competitors

1.3.1 JobSpy (speedyapply/JobSpy)

- **Stars:** 4.4k · **Language:** Python · **Last active:** Feb 2026
- **Coverage:** LinkedIn, Indeed, Glassdoor, Google Jobs, ZipRecruiter, Bayt, Naukri, BDJobs
- **Deployment:** `pip install python-jobspy`, returns a pandas DataFrame
- **State tracking:** none — it’s a library, not an app
- **Scoring:** none
- **Target:** developers building their own aggregator

Verdict: The de-facto library for *consumer boards*. Where this repo focuses on ATS-native boards (where the hiring companies actually post first), JobSpy focuses on the aggregators (where the same job appears late and sometimes reposted). The two are complementary, not competitors.

1.3.2 ats-scrapers / jobhive

- **Coverage:** 50+ ATS platforms (Greenhouse, Lever, Ashby, Workday, ...)
- **Deployment:** library, no API key required, 27 columns of output
- **State tracking:** none
- **Scoring:** none
- **Target:** data engineers wanting raw ATS feeds

Verdict: The closest technical sibling. It covers the same universe of ATS platforms this repo does, but exposes them as a pipeline — no UI, no per-user state. If this repo had to “stand on the shoulders of giants” to go generic, ats-scrapers would be the dependency to adopt.

1.3.3 JobSentinel (cboyd0319/JobSentinel)

- **Coverage:** multiple boards + ATS, with ghost-job detection
- **Deployment:** local desktop workspace, Python + TypeScript
- **State tracking:** yes — save, notes, contacts, reminders, offers
- **Scoring:** preference-based (salary floor, keyword match) — the v1.3 release docs explicitly flag “Extract skills from description” as deferred pending AI integration
- **Target:** job seeker who wants a private, offline-first workflow

Verdict: The product closest in *spirit* to this repo. Shares the “private by default, local only, no telemetry” axis. Where this repo has invested is **LLM scoring already working** (both Ollama and Claude paths) and **big-tech-specific seniority badges** (L5/L6, E5/E6, IC7, ...). JobSentinel compensates with a richer UX around applications, reminders, and ghost-job scoring — this repo has none of that.

1.3.4 JobAggregator (SammyUrfen/JobAggregator)

- **Deployment:** FastAPI dashboard
- **Coverage:** job-board scrapers + remote-work APIs + company ATS
- **State tracking:** dedupe + expiration (jobs that disappear from a board are marked expired)

- **Scoring:** none visible
- **Target:** remote-first / India-based systems + AI roles

Verdict: Shows a reasonable packaging path: FastAPI + themed dashboard. The dedupe-across-sources + expire-when-removed logic is analogous to this repo's `job_index.json` + orphan rendering.

1.3.5 Find-Me-Job (MohamedMamdouh18/Find-Me-Job)

- **Coverage:** LinkedIn, RemoteOK (daily scrape)
- **Deployment:** self-hosted pipeline → Notion database + Telegram
- **State tracking:** yes, via Notion
- **Scoring: LLM-based (0-100) against CV**, auto-generates tailored cover letters for high-scoring matches
- **Target:** individuals who want end-to-end automation up to the “we drafted the cover letter for you” step

Verdict: The only project found that uses an LLM for fit scoring, like this repo does. The novelty is the auto-cover-letter step — a reasonable next feature for this repo if the user wants to go that way.

1.3.6 JobSpy-API (rainmanjam/jobspy-api)

- **Stars:** 379 · **Deployment:** Dockerized, HTTP API, API-key auth, rate limiting, proxy support
- **Coverage:** inherits JobSpy's boards
- **Target:** teams wanting to call a shared backend

Verdict: The reference “how to productize an open-source scraper” project. The HTTP + proxies + rate limits pattern is what this repo would need before hosting a SaaS variant.

1.3.7 job-board-aggregator (Feashliaa/)

- **Scale:** 1,000,000+ positions · 20,000+ companies
- **Platforms:** Greenhouse, Lever, Ashby, Workday, iCIMS, BambooHR, Paylocity
- **Deployment:** GitHub Actions ETL + client-side filterable search
- **Workers tuned per platform:** 50 for Workday, 30 for GH/Lever/iCIMS, 10 for BambooHR, 5 for Ashby/Paylocity

Verdict: The scale this repo could reach if it stopped curating 147 boards and started indexing every board on each ATS. The tradeoff: coverage vs. signal. This repo's 147 are hand-picked for the user's industries (cybersecurity, big-tech, music tech); Feashliaa's 20k companies include fast-food chains and gyms.

1.4 Commercial job-search CRMs (adjacent category)

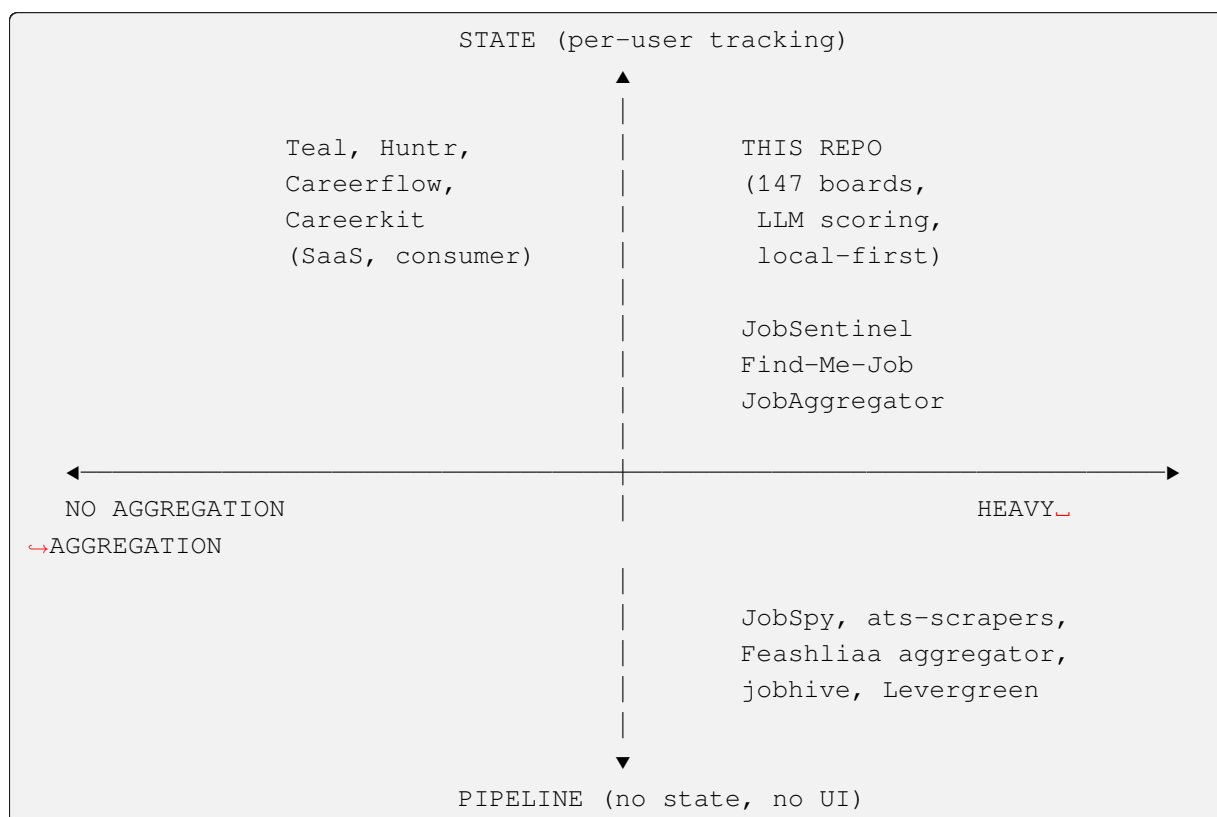
Not direct competitors — different buyer — but worth knowing about:

- **Huntr** — purpose-built job-search CRM, Chrome extension to one-click save postings, AI resume + cover letter writer
- **Teal** — bookmarked → applied → interviewing → offer stages, Chrome extension pulling from 40+ boards

- **Careerflow.ai** — LinkedIn profile audit + CRM + outbound tracking
- **Careerkit** — custom status columns, attach the resume version you submitted per entry
- **JibberJobber** — the “veteran” option, broad job-search management

All of these are web SaaS, require signup, and are optimized for the *application tracking* phase. This repo’s distinction is that it operates one phase earlier: **surfacing postings worth considering** across hundreds of career pages at once.

1.5 Where this repo sits on the market map



The upper-right quadrant — **aggregation + state + LLM scoring + senior-level bias** — is sparsely populated. This repo lives there. Its closest neighbours are JobSentinel (same UX bias, less aggregation) and Find-Me-Job (same LLM-scoring bias, far less aggregation).

1.6 Design decisions that are rare in the field

Based on this survey, the following choices in this repo are either unique or very rare among open-source competitors:

1. **Per-company dedicated fetchers alongside the generic ATS fetchers.** Most projects use the generic Greenhouse/Ashby/Workday APIs. This repo has dedicated code paths for Cisco (`fetch_cisco`), Bose (`fetch_bose`), IBM (`fetch_ibm`), LinkedIn (public guest pages), Apple, Google, Meta, Microsoft, NVIDIA — because those sources either don’t expose a clean API or have schema quirks the generic fetcher misses. This is maintenance cost but it’s also the reason the repo has signal where competitors have blanks.
2. **Big-tech-specific seniority mapping.** `SENIORITY_XP[Google][L7] → "~13y (Senior Staff)"`, `IC_LEVEL_XP[Netflix][L6] → "~9y (Staff)"`. No other open-source project

surveyed does this.

3. **Spontaneous-application surfacing.** The ☒ Spontaneous link on every company section lets the user send a cold application when the current board has nothing matching. No competitor does this.
4. **Score + reason + persistent cache.** Scores from the LLM get saved to `claude_fit_cache.json` with a timestamp, and the paste bar updates them in place without losing older scores.
5. **In-browser R/[?]/C controls that POST to a local HTTP server.** Classical SPAs talk to a hosted backend. This repo’s UI talks to a `127.0.0.1:8765` HTTP server co-located with the Python process. Simpler, no auth, no CORS pain — only works for a single user.

1.7 Realistic paths forward

1.7.1 Path A — Open-source as-is, no hosting

- Publish on GitHub with a self-hosted “quick start” README
- Target: senior engineers who already run a dev environment
- Install burden: `git clone + pip install playwright + python3 jobs.py`
- Cost: zero
- Distribution: GitHub topic tagging (self-hosted, job-search, job-aggregator) + mention on `r/cscareerquestions` + HN “Show HN”

Difficulty: Low. Mostly a refactor pass (which the user has already asked for: separate engine from data, example config, etc.).

1.7.2 Path B — Dockerized self-hosted

- Package Playwright + Python + the engine as a `docker compose up`
- Target: homelabbers (same audience as Immich / Paperless-ngx)
- Install burden: `edit user_config.py, mount ./data, docker up`
- Cost: zero to the user
- Distribution: AwesomeSelfHosted list, `r/selfhosted`, LinuxServer

Difficulty: Medium. Playwright headless Chromium in Docker is a known-painful area (apt packages, font issues, image size ~1.5 GB). The engine already runs headless so no architectural change — only packaging.

1.7.3 Path C — SaaS

- Multi-tenant, cloud-hosted, pay-per-month
- Target: engineers who don’t want to self-host anything

Blockers:

- **Legal.** LinkedIn / Google / Apple scrape protections. *hiQ v. LinkedIn* (2022) narrowed the “public data” shield; a hosted service that scrapes LinkedIn on behalf of third parties is clearly riskier than running the same code on your own laptop with your own IP.
- **Anti-bot.** Datacenter IPs get flagged by LinkedIn within hours. Residential proxies cost \$5-15/GB and would blow the margin on any reasonable subscription price.

- **Playwright at scale.** 147 boards × Chromium per fetch × N users is expensive compute. A full run on the user's Mac takes 200s with parallelism and warm cache.
- **State isolation.** JSON state files become multi-tenant SQL. UI grows auth, billing, account management.

Difficulty: High. The engineering is doable but the regulatory + cost + anti-bot story is a sustained headwind.

1.7.4 Path D — “Managed self-hosted”

- The hosted service helps the user DEPLOY the self-hosted version (one-click deploy to Fly.io / Railway / Render)
- The user owns the IP, the data, and the risk
- The service collects a one-time setup fee + optional support

Difficulty: Low once Path B is done. The pattern is `railway.app/template/<yourtemplate>` with the user's env vars.

1.8 Recommendation

Go Path A → Path B. Open-source first, Docker second. SaaS is a legal + operational minefield that extracts value this project doesn't need to capture.

Specifically:

1. Finish the refactor the user already started (engine/data separation, delete `PROFILE.md`, `user_config.example.py`).
2. Write a 15-line README showing a copy-paste quick-start.
3. Push to GitHub, tag with topics, Show HN once.
4. Add a `Dockerfile` + `docker-compose.yml` in the following month.
5. Add a one-click deploy template to Fly.io for users who want remote-but-still-self-hosted.
6. Skip SaaS entirely. Point users who ask for one to Huntr / Teal for the application-tracking side and keep this repo as the aggregation + scoring side.

1.9 Open questions worth asking the user

- Target audience: senior engineers only, or do you want it to work equally well for a mid-level or a product manager?
- SENIORITY_XP opinionatedness: these are tuned for “~5-15 years into tech”. Someone who's 25 years in won't recognize themselves in the grid.
- Industry breadth: 147 companies currently skew cybersecurity / big-tech / music-tech. For a generic release, a `--industry cybersecurity` CLI flag + bundled industry configs would reduce onboarding friction.
- Hosting ambition: are you optimizing for your own hunt only, or genuinely wanting to help others?